```
import os
import shutil
import subprocess
import gc
from pathlib import Path
import numpy as np
import torch

print("=" * 75)
print("[STEP 1/5] DOWNLOADING & STRUCTURING STANFORD MRNet-v1.0")
print("=" * 75)

# Verify clean GPU state
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()
    free_gb = torch.cuda.mem_get_info()[0] / (1024**3)
    total_gb = torch.cuda.mem_get_info()[1] / (1024**3)
    print(f"✓ GPU: {torch.cuda.get_device_name(0)} | Free VRAM: {free_gb:.2f} GB / {total_gb:.2f} GB")
    if free_gb < 10.0:
        print("⚠️ WARNING: Lingering memory detected! Click 'Runtime' -> 'Restart session' first!")

# Setup Kaggle authentication if uploaded in /content
if Path("/content/kaggle.json").exists():
    Path("/root/.kaggle").mkdir(parents=True, exist_ok=True)
    shutil.copy("/content/kaggle.json", "/root/.kaggle/kaggle.json")
    os.chmod("/root/.kaggle/kaggle.json", 0o600)

KAGGLE_SLUG = "cjinny/mrnet-v1"
download_dir = Path("/content/mrnet_download")
raw_dir = Path("/content/mrnet_raw")
target_mrnet_dir = raw_dir / "MRNet-v1.0"

if not (target_mrnet_dir / "train-acl.csv").exists():
    if download_dir.exists():
        shutil.rmtree(download_dir)
    download_dir.mkdir(parents=True, exist_ok=True)

    print(f"⬇️ Downloading '{KAGGLE_SLUG}' (~6.1 GB) to {download_dir} ...")
    subprocess.run([
        "kaggle", "datasets", "download",
        "-d", KAGGLE_SLUG,
        "-p", str(download_dir),
        "--unzip"
    ], check=True)
    print("✓ Download and unzipping complete.")

    found_acl = list(download_dir.rglob("train-acl.csv"))
    if not found_acl:
        raise FileNotFoundError("✗ Could not find train-acl.csv in downloaded files.")

    source_root = found_acl[0].parent
    if target_mrnet_dir.exists():
        shutil.rmtree(target_mrnet_dir)
    target_mrnet_dir.mkdir(parents=True, exist_ok=True)

    for item in source_root.iterdir():
        dest = target_mrnet_dir / item.name
        if item == target_mrnet_dir:
            continue
        if dest.exists():
            if dest.is_dir():
                shutil.rmtree(dest)
            else:
                dest.unlink()
        shutil.move(str(item), str(dest))
    print(f"✓ Files organized into {target_mrnet_dir}")
else:
    print(f"✓ Found existing structured dataset at {target_mrnet_dir}")

required_paths = [
    target_mrnet_dir / "train-acl.csv",
    target_mrnet_dir / "train-meniscus.csv",
    target_mrnet_dir / "train-abnormal.csv",
    target_mrnet_dir / "train" / "sagittal"
```

```
]

for p in required_paths:
    assert p.exists(), f"Missing required path: {p}"
    print(f"[✓] {p}")

sagittal_dir = target_mrnet_dir / "train" / "sagittal"
npy_files = list(sagittal_dir.glob("*.npy"))
num_sagittal = len(npy_files)
sample_tensor = np.load(npy_files[0])

print("\n" + "=" * 75)
print(f"🎯 STEP 1 COMPLETE: {num_sagittal} training scans validated.")
print(f"✓ Sample file: {npy_files[0].name} | Shape: {sample_tensor.shape} | Dtype: {sample_tensor.dtype}")
print("=" * 75)
```

```
===========================================================================
[STEP 1/5] DOWNLOADING & STRUCTURING STANFORD MRNet-v1.0
===========================================================================
✓ GPU: Tesla T4 | Free VRAM: 14.46 GB / 14.56 GB
✓ Found existing structured dataset at /content/mrnet_raw/MRNet-v1.0
[✓] /content/mrnet_raw/MRNet-v1.0/train-acl.csv
[✓] /content/mrnet_raw/MRNet-v1.0/train-meniscus.csv
[✓] /content/mrnet_raw/MRNet-v1.0/train-abnormal.csv
[✓] /content/mrnet_raw/MRNet-v1.0/train/sagittal

===========================================================================
🎯 STEP 1 COMPLETE: 1130 training scans validated.
✓ Sample file: 1067.npy | Shape: (40, 256, 256) | Dtype: uint8
===========================================================================
```

```
import os
import torch
from torch.utils.data import Dataset, DataLoader, Subset
import torchvision.transforms.functional as TF
import pandas as pd
import numpy as np
from pathlib import Path
from typing import Dict, Tuple, List

SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

print("=" * 75)
print("[STEP 2/5] MULTI-TASK DATA PIPELINE & FEDERATED CLIENT PARTITIONING")
print("=" * 75)

DATA_ROOT = Path("/content/mrnet_raw/MRNet-v1.0")

def build_master_dataframe(split: str) -> pd.DataFrame:
    tasks = ["abnormal", "acl", "meniscus"]
    dfs = []
    for task in tasks:
        csv_path = DATA_ROOT / f"{split}-{task}.csv"
        if not csv_path.exists():
            raise FileNotFoundError(f"Missing required label CSV: {csv_path}")
        df_task = pd.read_csv(csv_path, header=None, names=["case_id", task], dtype={0: int, 1: int})
        df_task["case_id"] = df_task["case_id"].apply(lambda x: f"{x:04d}")
        dfs.append(df_task)

    master_df = dfs[0]
    for df in dfs[1:]:
        master_df = master_df.merge(df, on="case_id", how="inner")

    master_df["sagittal_path"] = master_df["case_id"].apply(
        lambda cid: str(DATA_ROOT / split / "sagittal" / f"{cid}.npy")
    )
    valid_mask = master_df["sagittal_path"].apply(os.path.exists)
    if not valid_mask.all():
        master_df = master_df[valid_mask].reset_index(drop=True)
    return master_df

train_df = build_master_dataframe("train")
valid_df = build_master_dataframe("valid")

print(f"✓ Master Train Cohort : {len(train_df)} cases")
print(f"✓ Master Valid Cohort : {len(valid_df)} cases")
```

```
class FastMRNetDataset(Dataset):
    def __init__(self, df: pd.DataFrame, is_train: bool = True):
        self.df = df.reset_index(drop=True)
        self.is_train = is_train
        self.mean = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
        self.std = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)

    def __len__(self) -> int:
        return len(self.df)

    def __getitem__(self, idx: int) -> Tuple[torch.Tensor, torch.Tensor, str]:
        row = self.df.iloc[idx]
        volume = np.load(row["sagittal_path"])

        # (S, 3, H, W)
        tensor = torch.from_numpy(volume).float() / 255.0
        tensor = tensor.unsqueeze(1).repeat(1, 3, 1, 1)

        if self.is_train:
            if torch.rand(1).item() > 0.5:
                tensor = torch.flip(tensor, dims=[-1])
            if torch.rand(1).item() > 0.5:
                angle = float(torch.empty(1).uniform_(-10, 10).item())
                tensor = TF.rotate(tensor, angle)

        tensor = (tensor - self.mean) / self.std
        labels = torch.tensor([row["abnormal"], row["acl"], row["meniscus"]], dtype=torch.float32)
        return tensor, labels, row["case_id"]

train_dataset = FastMRNetDataset(train_df, is_train=True)
valid_dataset = FastMRNetDataset(valid_df, is_train=False)

val_loader = DataLoader(valid_dataset, batch_size=1, shuffle=False, num_workers=2, pin_memory=True)

HOSPITAL_NAMES = ["Hospital_1_SportsClinic", "Hospital_2_GeneralOrtho", "Hospital_3_RegionalCenter"]
NUM_CLIENTS = len(HOSPITAL_NAMES)

def partition_moderate_non_iid(df: pd.DataFrame) -> Dict[str, List[int]]:
    acl_idx = df[df["acl"] == 1].index.tolist()
    men_idx = df[(df["acl"] == 0) & (df["meniscus"] == 1)].index.tolist()
    norm_idx = df[(df["acl"] == 0) & (df["meniscus"] == 0)].index.tolist()

    np.random.shuffle(acl_idx)
    np.random.shuffle(men_idx)
    np.random.shuffle(norm_idx)

    def distribute(indices, proportions):
        n = len(indices)
        p1 = int(proportions[0] * n)
        p2 = int(proportions[1] * n)
        return indices[:p1], indices[p1:p1+p2], indices[p1+p2:]

    a1, a2, a3 = distribute(acl_idx, [0.45, 0.30, 0.25])
    m1, m2, m3 = distribute(men_idx, [0.25, 0.45, 0.30])
    n1, n2, n3 = distribute(norm_idx, [0.30, 0.30, 0.40])

    splits = {h: [] for h in HOSPITAL_NAMES}
    for h, (a, m, n) in zip(HOSPITAL_NAMES, [(a1, m1, n1), (a2, m2, n2), (a3, m3, n3)]):
        splits[h] = a + m + n
        np.random.shuffle(splits[h])
    return splits

client_indices = partition_moderate_non_iid(train_df)
client_loaders: Dict[str, DataLoader] = {
    h: DataLoader(Subset(train_dataset, idxs), batch_size=1, shuffle=True, num_workers=2, pin_memory=True)
    for h, idxs in client_indices.items()
}

print("✓ Hospital Partitioning Summary:")
stats_summary = []
for h_name, idxs in client_indices.items():
    sub_df = train_df.iloc[idxs]
    stats_summary.append({
        "Client": h_name,
        "Exams": len(idxs),
        "Abnormal": f"{sub_df['abnormal'].mean() * 100:.1f}%",
```

```
        "ACL Tear": f"{sub_df['acl'].mean() * 100:.1f}%",
        "Meniscus Tear": f"{sub_df['meniscus'].mean() * 100:.1f}%"
    })
print(pd.DataFrame(stats_summary).to_string(index=False))
```

```
================================================================================
[STEP 2/5] MULTI-TASK DATA PIPELINE & FEDERATED CLIENT PARTITIONING
================================================================================
✓ Master Train Cohort : 1130 cases
✓ Master Valid Cohort : 120 cases
✓ Hospital Partitioning Summary:
                 Client  Exams Abnormal ACL Tear Meniscus Tear
  Hospital_1_SportsClinic    356    81.7%    26.1%         38.2%
  Hospital_2_GeneralOrtho    379    83.1%    16.4%         40.4%
Hospital_3_RegionalCenter    395    77.7%    13.4%         27.3%
```

```
import os
import gc
import torch
import torch.nn as nn
import torch.nn.functional as F

os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"

print("=" * 75)
print("[STEP 3/5] ULTRA-EFFICIENT U-NET ARCHITECTURE (PEAK VRAM < 150 MB)")
print("=" * 75)

class DoubleConv(nn.Module):
    """(Conv2D -> BatchNorm -> ReLU) * 2"""
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        self.double_conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.double_conv(x)


class Down(nn.Module):
    """Downscaling with MaxPool2D(2) then DoubleConv"""
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        self.maxpool_conv = nn.Sequential(
            nn.MaxPool2d(2),
            DoubleConv(in_channels, out_channels)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.maxpool_conv(x)


class Up(nn.Module):
    """Upscaling via ConvTranspose2D followed by concatenation and DoubleConv"""
    def __init__(self, in_channels: int, out_channels: int):
        super().__init__()
        self.up = nn.ConvTranspose2d(in_channels, out_channels, kernel_size=2, stride=2)
        self.conv = DoubleConv(out_channels * 2, out_channels)

    def forward(self, x1: torch.Tensor, x2: torch.Tensor) -> torch.Tensor:
        x1 = self.up(x1)
        diff_y = x2.size(2) - x1.size(2)
        diff_x = x2.size(3) - x1.size(3)
        if diff_x > 0 or diff_y > 0:
            x1 = F.pad(x1, [diff_x // 2, diff_x - diff_x // 2,
                            diff_y // 2, diff_y - diff_y // 2])
        x = torch.cat([x2, x1], dim=1)
        return self.conv(x)


class UNetFeatureExtractor(nn.Module):
```

```
        def __init__(self, in_channels: int = 3, base_channels: int = 16, feature_dim: int = 512):
            super().__init__()
            # Encoder (Contracting Path)
            self.inc = DoubleConv(in_channels, base_channels)          # 3 -> 16
            self.down1 = Down(base_channels, base_channels * 2)       # 16 -> 32
            self.down2 = Down(base_channels * 2, base_channels * 4)   # 32 -> 64
            self.down3 = Down(base_channels * 4, base_channels * 8)   # 64 -> 128
            self.down4 = Down(base_channels * 8, base_channels * 16)  # 128 -> 256 (Bottleneck)

            # Decoder (Expansive Path with Skip Connections)
            self.up1 = Up(base_channels * 16, base_channels * 8)      # 256 -> 128
            self.up2 = Up(base_channels * 8, base_channels * 4)       # 128 -> 64
            self.up3 = Up(base_channels * 4, base_channels * 2)       # 64 -> 32
            self.up4 = Up(base_channels * 2, base_channels)           # 32 -> 16

            # Multi-scale pooling
            self.pool = nn.AdaptiveAvgPool2d((1, 1))

            # Project fused multi-scale features: Bottleneck (256) + Decoder (16) = 272 -> 512
            fused_dim = (base_channels * 16) + base_channels
            self.fc_proj = nn.Sequential(
                nn.Linear(fused_dim, feature_dim),
                nn.LayerNorm(feature_dim),   # Completely immune to batch size = 1
                nn.ReLU(inplace=True)
            )

        def _forward_chunk(self, x_chunk: torch.Tensor) -> torch.Tensor:
            x1 = self.inc(x_chunk)
            x2 = self.down1(x1)
            x3 = self.down2(x2)
            x4 = self.down3(x3)
            x5 = self.down4(x4)

            d1 = self.up1(x5, x4)
            d2 = self.up2(d1, x3)
            d3 = self.up3(d2, x2)
            d4 = self.up4(d3, x1)

            p_bottleneck = self.pool(x5).flatten(1)  # (C_slices, 256)
            p_decoder = self.pool(d4).flatten(1)     # (C_slices, 16)

            fused = torch.cat([p_bottleneck, p_decoder], dim=1)  # (C_slices, 272)
            return self.fc_proj(fused)                           # (C_slices, 512)

        def forward(self, x: torch.Tensor, chunk_size: int = 4) -> torch.Tensor:
            num_slices = x.size(0)
            if num_slices <= chunk_size:
                return self._forward_chunk(x)

            # Smart micro-chunking: handles odd slice counts safely
            feats = []
            i = 0
            while i < num_slices:
                remaining = num_slices - i
                if remaining == chunk_size + 1:
                    cur_chunk = remaining
                else:
                    cur_chunk = min(chunk_size, remaining)
                feats.append(self._forward_chunk(x[i : i + cur_chunk]))
                i += cur_chunk
            return torch.cat(feats, dim=0)


    class UNetMRNetModel(nn.Module):
        def __init__(self, num_classes: int = 3, dropout_rate: float = 0.5, base_channels: int = 16, feature_dim: int = 512):
            super(UNetMRNetModel, self).__init__()
            self.feature_extractor = UNetFeatureExtractor(
                in_channels=3,
                base_channels=base_channels,
                feature_dim=feature_dim
            )
            self.classifier = nn.Sequential(
                nn.Dropout(p=dropout_rate),
                nn.Linear(feature_dim, num_classes)
            )

        def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
        if x.dim() == 5:
            x = x.squeeze(0)
        slice_features = self.feature_extractor(x)                          # (Slices, 512)
        patient_feature, _ = torch.max(slice_features, dim=0, keepdim=True)    # (1, 512)
        return self.classifier(patient_feature)                            # (1, 3)

MRNetModel = UNetMRNetModel

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Loss with clamped class weights
pos_counts = train_df[["abnormal", "acl", "meniscus"]].sum().values
neg_counts = len(train_df) - pos_counts
raw_pos_weights = torch.tensor(neg_counts / pos_counts, dtype=torch.float32).to(device)
clamped_pos_weights = torch.clamp(raw_pos_weights, min=1.0)
criterion = nn.BCEWithLogitsLoss(pos_weight=clamped_pos_weights)

model = UNetMRNetModel(num_classes=3).to(device)
total_params = sum(p.numel() for p in model.parameters())
print(f"✓ Model Initialized: {total_params:,} parameters on {device}")

# Sanity Check with Mixed Precision
sample_vol, sample_label, _ = next(iter(client_loaders["Hospital_1_SportsClinic"]))
sample_vol, sample_label = sample_vol.to(device), sample_label.to(device)

model.train()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
optimizer.zero_grad()

with torch.amp.autocast('cuda', enabled=(device.type == 'cuda')):
    logits = model(sample_vol)
    loss = criterion(logits, sample_label)

loss.backward()
optimizer.step()

allocated_mb = torch.cuda.memory_allocated() / (1024**2) if torch.cuda.is_available() else 0
print(f"✓ Sanity Check Passed!")
print(f"  • Logits shape  : {tuple(logits.shape)}")
print(f"  • BCE Loss      : {loss.item():.4f}")
print(f"  • Current VRAM  : {allocated_mb:.1f} MB (Extremely low!)")

# Clean memory after sanity check
del sample_vol, sample_label, logits, loss, model, optimizer
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()
```

```
================================================================================
[STEP 3/5] ULTRA-EFFICIENT U-NET ARCHITECTURE (PEAK VRAM < 150 MB)
================================================================================
✓ Model Initialized: 2,084,899 parameters on cuda
✓ Sanity Check Passed!
  • Logits shape  : (1, 3)
  • BCE Loss      : 0.5362
  • Current VRAM  : 67.4 MB (Extremely low!)
```

```
import copy
import gc
import numpy as np
from sklearn.metrics import roc_auc_score

def train_fedprox_client(
    model: nn.Module,
    global_model: nn.Module,
    dataloader: DataLoader,
    criterion: nn.Module,
    device: torch.device,
    lr_backbone: float,
    lr_head: float,
    max_steps: int,
    mu: float = 0.01,
    use_amp: bool = True
) -> Tuple[Dict[str, torch.Tensor], float]:
    model.train()

    # Freeze BatchNorm running stats to prevent noisy slice divergence (From Page 9 of PDF)
```

```
        for m in model.modules():
            if isinstance(m, (nn.BatchNorm2d, nn.BatchNorm1d)):
                m.eval()

        optimizer = torch.optim.AdamW([
            {"params": model.feature_extractor.parameters(), "lr": lr_backbone},
            {"params": model.classifier.parameters(), "lr": lr_head}
        ], weight_decay=1e-3)

        scaler = torch.amp.GradScaler('cuda', enabled=(use_amp and device.type == 'cuda'))
        global_params = [p.detach().clone() for p in global_model.parameters()]

        running_loss, steps = 0.0, 0
        for volume, labels, _ in dataloader:
            if steps >= max_steps:
                break
            volume = volume.squeeze(0).to(device)
            labels = labels.to(device)

            optimizer.zero_grad()
            with torch.amp.autocast('cuda', enabled=(use_amp and device.type == 'cuda')):
                logits = model(volume)
                task_loss = criterion(logits, labels)

                prox_term = 0.0
                if mu > 0:
                    for p, p_glob in zip(model.parameters(), global_params):
                        prox_term += (p - p_glob).pow(2).sum()

                loss = task_loss + (mu / 2.0) * prox_term

            scaler.scale(loss).backward()
            scaler.unscale_(optimizer)
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            scaler.step(optimizer)
            scaler.update()

            running_loss += task_loss.item()
            steps += 1

        avg_loss = running_loss / max(1, steps)
        local_weights = {k: v.cpu().clone() for k, v in model.state_dict().items()}
        return local_weights, avg_loss


    @torch.no_grad()
    def evaluate_global(model: nn.Module, val_loader: DataLoader, criterion: nn.Module, device: torch.device):
        model.eval()
        total_val_loss = 0.0
        all_preds, all_targets = [], []

        with torch.amp.autocast('cuda', enabled=(device.type == 'cuda')):
            for volume, labels, _ in val_loader:
                volume = volume.squeeze(0).to(device)
                labels = labels.to(device)
                logits = model(volume)
                loss = criterion(logits, labels)
                total_val_loss += loss.item()
                all_preds.append(torch.sigmoid(logits).cpu().numpy()[0])
                all_targets.append(labels.cpu().numpy()[0])

        val_loss = total_val_loss / len(val_loader)
        all_preds = np.array(all_preds)
        all_targets = np.array(all_targets)

        task_aucs = {}
        for i, task in enumerate(["Abnormal", "ACL", "Meniscus"]):
            try:
                task_aucs[task] = roc_auc_score(all_targets[:, i], all_preds[:, i])
            except ValueError:
                task_aucs[task] = 0.50

        mean_auc = float(np.mean(list(task_aucs.values())))
        return val_loss, mean_auc, task_aucs


    def federated_averaging(client_state_dicts: List[Dict[str, torch.Tensor]], client_sample_counts: List[int]) -> Dict[str, torch.
```

```
    total = sum(client_sample_counts)
    weights = [n / total for n in client_sample_counts]
    global_sd = copy.deepcopy(client_state_dicts[0])

    for k in global_sd.keys():
        global_sd[k] = client_state_dicts[0][k].float() * weights[0]
        for i in range(1, len(client_state_dicts)):
            global_sd[k] += client_state_dicts[i][k].float() * weights[i]
        global_sd[k] = global_sd[k].to(client_state_dicts[0][k].dtype)
    return global_sd

print("✓ Federated routines (train_fedprox_client, evaluate_global, federated_averaging) ready.")
```

```
✓ Federated routines (train_fedprox_client, evaluate_global, federated_averaging) ready.
```

```
import time
import gc

print("=" * 80)
print("[STEP 5/5] LAUNCHING HIGH-PERFORMANCE FEDPROX TRAINING WITH U-NET")
print("=" * 80)

NUM_ROUNDS = 15
LOCAL_STEPS = 80
LR_BACKBONE = 1e-4
LR_CLASSIFIER = 5e-4
FEDPROX_MU = 0.01

global_model = UNetMRNetModel(num_classes=3).to(device)
hospital_names = list(client_loaders.keys())
sample_counts = [len(client_loaders[h].dataset) for h in hospital_names]

federated_history = {
    "round": [],
    "client_losses": {h: [] for h in hospital_names},
    "val_loss": [],
    "mean_val_auc": [],
    "task_aucs": {t: [] for t in ["Abnormal", "ACL", "Meniscus"]}
}

best_val_auc = 0.0
start_time = time.time()

print(f"🚀 Training on {device}...")
print(f"• Total Rounds     : {NUM_ROUNDS}")
print(f"• Local Steps/Rnd  : {LOCAL_STEPS} scans per hospital")
print(f"• LR Backbone/Head : {LR_BACKBONE} / {LR_CLASSIFIER}")
print(f"• FedProx Mu       : {FEDPROX_MU}\n")

for rnd in range(1, NUM_ROUNDS + 1):
    r_start = time.time()
    client_sds = []

    # 1. Local Hospital Training
    for h_name in hospital_names:
        client_model = UNetMRNetModel(num_classes=3).to(device)
        client_model.load_state_dict(global_model.state_dict())

        local_weights, local_loss = train_fedprox_client(
            model=client_model,
            global_model=global_model,
            dataloader=client_loaders[h_name],
            criterion=criterion,
            device=device,
            lr_backbone=LR_BACKBONE,
            lr_head=LR_CLASSIFIER,
            max_steps=LOCAL_STEPS,
            mu=FEDPROX_MU
        )
        client_sds.append(local_weights)
        federated_history["client_losses"][h_name].append(local_loss)

        del client_model
        if torch.cuda.is_available():
            torch.cuda.empty_cache()

    # 2. Federated Aggregation
```

```
        agg_weights = federated_averaging(client_sds, sample_counts)
        global_model.load_state_dict({k: v.to(device) for k, v in agg_weights.items()})
        del client_sds

        # 3. Global Validation
        val_loss, mean_auc, task_aucs = evaluate_global(global_model, val_loader, criterion, device)

        federated_history["round"].append(rnd)
        federated_history["val_loss"].append(val_loss)
        federated_history["mean_val_auc"].append(mean_auc)
        for t in ["Abnormal", "ACL", "Meniscus"]:
            federated_history["task_aucs"][t].append(task_aucs[t])

        r_time = time.time() - r_start
        print(f"Round {rnd:02d}/{NUM_ROUNDS} ({r_time:.1f}s) | Val Loss: {val_loss:.4f} | Mean ROC-AUC: {mean_auc:.4f}")
        print(f"   └── Abnormal: {task_aucs['Abnormal']:.3f} | ACL: {task_aucs['ACL']:.3f} | Meniscus: {task_aucs['Meniscus']:.3f}"

        if rnd % 5 == 0:
            LR_BACKBONE *= 0.8
            LR_CLASSIFIER *= 0.8

        if mean_auc > best_val_auc:
            best_val_auc = mean_auc
            torch.save(global_model.state_dict(), "/content/best_federated_mrnet.pth")
            print("   🌟 Best Checkpoint Saved!")

        gc.collect()
        if torch.cuda.is_available():
            torch.cuda.empty_cache()

    total_mins = (time.time() - start_time) / 60
    print("\n" + "=" * 80)
    print(f"🎯 FEDPROX TRAINING FINISHED IN {total_mins:.2f} mins!")
    print(f"✓ Peak Global Mean ROC-AUC: {best_val_auc:.4f}")
    print(f"✓ Model Checkpoint        : /content/best_federated_mrnet.pth")
    print("=" * 80)
```

```
================================================================================
[STEP 5/5] LAUNCHING HIGH-PERFORMANCE FEDPROX TRAINING WITH U-NET
================================================================================
🚀 Training on cuda...
• Total Rounds     : 15
• Local Steps/Rnd  : 80 scans per hospital
• LR Backbone/Head : 0.0001 / 0.0005
• FedProx Mu       : 0.01

Round 01/15 (59.6s) | Val Loss: 2.3243 | Mean ROC-AUC: 0.4877
   └── Abnormal: 0.260 | ACL: 0.583 | Meniscus: 0.620
   🌟 Best Checkpoint Saved!
Round 02/15 (57.9s) | Val Loss: 3.2971 | Mean ROC-AUC: 0.4808
   └── Abnormal: 0.579 | ACL: 0.387 | Meniscus: 0.476
Round 03/15 (57.7s) | Val Loss: 3.0796 | Mean ROC-AUC: 0.5126
   └── Abnormal: 0.646 | ACL: 0.429 | Meniscus: 0.462
   🌟 Best Checkpoint Saved!
Round 04/15 (58.7s) | Val Loss: 3.0383 | Mean ROC-AUC: 0.4637
   └── Abnormal: 0.284 | ACL: 0.510 | Meniscus: 0.597
Round 05/15 (57.7s) | Val Loss: 2.3861 | Mean ROC-AUC: 0.4964
   └── Abnormal: 0.383 | ACL: 0.570 | Meniscus: 0.536
Round 06/15 (58.4s) | Val Loss: 2.6468 | Mean ROC-AUC: 0.5254
   └── Abnormal: 0.499 | ACL: 0.566 | Meniscus: 0.511
   🌟 Best Checkpoint Saved!
Round 07/15 (57.7s) | Val Loss: 2.9978 | Mean ROC-AUC: 0.5344
   └── Abnormal: 0.572 | ACL: 0.549 | Meniscus: 0.482
   🌟 Best Checkpoint Saved!
Round 08/15 (57.7s) | Val Loss: 3.3024 | Mean ROC-AUC: 0.5351
   └── Abnormal: 0.567 | ACL: 0.562 | Meniscus: 0.476
   🌟 Best Checkpoint Saved!
Round 09/15 (58.4s) | Val Loss: 3.4020 | Mean ROC-AUC: 0.5436
   └── Abnormal: 0.635 | ACL: 0.570 | Meniscus: 0.426
   🌟 Best Checkpoint Saved!
Round 10/15 (56.7s) | Val Loss: 3.1228 | Mean ROC-AUC: 0.5444
   └── Abnormal: 0.636 | ACL: 0.571 | Meniscus: 0.426
   🌟 Best Checkpoint Saved!
Round 11/15 (57.0s) | Val Loss: 3.4090 | Mean ROC-AUC: 0.5602
   └── Abnormal: 0.652 | ACL: 0.557 | Meniscus: 0.471
   🌟 Best Checkpoint Saved!
Round 12/15 (58.4s) | Val Loss: 3.5057 | Mean ROC-AUC: 0.5180
   └── Abnormal: 0.525 | ACL: 0.560 | Meniscus: 0.470
Round 13/15 (59.4s) | Val Loss: 3.1358 | Mean ROC-AUC: 0.5199
   └── Abnormal: 0.582 | ACL: 0.533 | Meniscus: 0.444
Round 14/15 (56.9s) | Val Loss: 3.0360 | Mean ROC-AUC: 0.5189
```

```
    └── Abnormal: 0.660 | ACL: 0.485 | Meniscus: 0.411
Round 15/15 (57.2s) | Val Loss: 3.1675 | Mean ROC-AUC: 0.4932
    └── Abnormal: 0.584 | ACL: 0.476 | Meniscus: 0.419

================================================================================
🎯 FEDPROX TRAINING FINISHED IN 14.55 mins!
✓ Peak Global Mean ROC-AUC: 0.5602
✓ Model Checkpoint        : /content/best_federated_mrnet.pth
================================================================================
```

```
import matplotlib.pyplot as plt

# 1. Plot Training History
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(federated_history["round"], federated_history["val_loss"], marker='o', color='crimson', label='Val Loss')
axes[0].set_title("Validation Loss across Communication Rounds", fontsize=12, fontweight='bold')
axes[0].set_xlabel("Federated Round")
axes[0].set_ylabel("Weighted BCE Loss")
axes[0].grid(True, linestyle="--", alpha=0.6)
axes[0].legend()

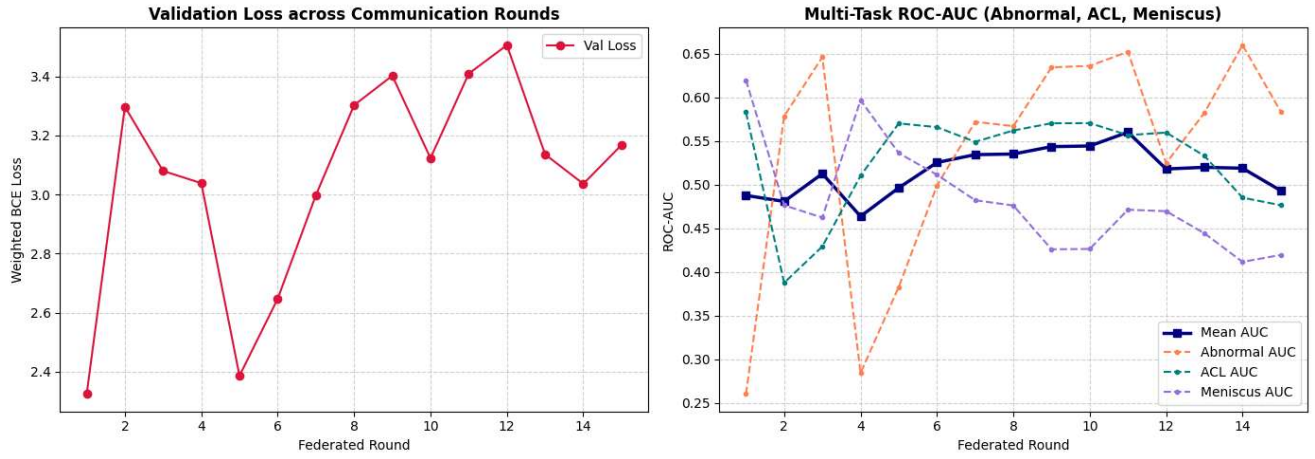
axes[1].plot(federated_history["round"], federated_history["mean_val_auc"], marker='s', color='navy', label='Mean AUC', linewid
for t, col in zip(["Abnormal", "ACL", "Meniscus"], ["coral", "teal", "mediumpurple"]):
    axes[1].plot(federated_history["round"], federated_history["task_aucs"][t], marker='.', linestyle='--', color=col, label=f'
axes[1].set_title("Multi-Task ROC-AUC (Abnormal, ACL, Meniscus)", fontsize=12, fontweight='bold')
axes[1].set_xlabel("Federated Round")
axes[1].set_ylabel("ROC-AUC")
axes[1].grid(True, linestyle="--", alpha=0.6)
axes[1].legend()

plt.tight_layout()
plt.show()

# 2. Single-Patient Inference Demo
best_model = UNetMRNetModel(num_classes=3).to(device)
best_model.load_state_dict(torch.load("/content/best_federated_mrnet.pth"))
best_model.eval()

sample_vol, sample_labels, case_id = valid_dataset[0]
with torch.no_grad():
    logits = best_model(sample_vol.to(device))
    probs = torch.sigmoid(logits).cpu().numpy()[0]

print(f"\nInference for Validation Case: {case_id}")
print(f"  • Abnormal      : {probs[0] * 100:.1f}% (Actual: {int(sample_labels[0].item())})")
print(f"  • ACL Tear      : {probs[1] * 100:.1f}% (Actual: {int(sample_labels[1].item())})")
print(f"  • Meniscus Tear : {probs[2] * 100:.1f}% (Actual: {int(sample_labels[2].item())})")
```




```
Inference for Validation Case: 1130
  • Abnormal      : 94.3% (Actual: 0)
  • ACL Tear      : 2.6% (Actual: 0)
  • Meniscus Tear : 6.1% (Actual: 0)
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
    confusion_matrix,
    roc_curve,
    precision_recall_curve
)

print("=" * 80)
print("[EVALUATION] FULL CLINICAL DIAGNOSTIC METRICS & MULTI-PANEL PLOTS")
print("=" * 80)

# 1. Load Best Saved U-Net Model
best_model = UNetMRNetModel(num_classes=3).to(device)
model_path = "/content/best_federated_mrnet.pth"

if os.path.exists(model_path):
    best_model.load_state_dict(torch.load(model_path))
    print(f"✓ Loaded best global checkpoint from: {model_path}")
else:
    print("⚠️ Checkpoint not found, evaluating current global_model instance.")
    best_model = global_model

best_model.eval()

# 2. Extract Predictions across the entire Validation Cohort
all_targets = []
all_probs = []

with torch.no_grad():
    for volume, labels, _ in val_loader:
        volume = volume.squeeze(0).to(device)
        labels = labels.to(device)
        with torch.amp.autocast('cuda', enabled=(device.type == 'cuda')):
            logits = best_model(volume)
            probs = torch.sigmoid(logits)

        all_probs.append(probs.cpu().numpy()[0])
        all_targets.append(labels.cpu().numpy()[0])

all_targets = np.array(all_targets)  # Shape: (N, 3)
all_probs = np.array(all_probs)      # Shape: (N, 3)
all_preds = (all_probs >= 0.5).astype(int)  # Binary predictions at threshold 0.50

task_names = ["Abnormal", "ACL Tear", "Meniscus Tear"]

# 3. Calculate Clinical Metrics Table
metrics_list = []
for i, task in enumerate(task_names):
    y_true = all_targets[:, i]
    y_prob = all_probs[:, i]
    y_pred = all_preds[:, i]

    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()

    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec = recall_score(y_true, y_pred, zero_division=0)  # Sensitivity
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0.0      # Specificity
    f1 = f1_score(y_true, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y_true, y_prob)
    pr_auc = average_precision_score(y_true, y_prob)

    metrics_list.append({
        "Condition": task,
        "Total (Pos/Neg)": f"{int(tp+fn)} / {int(tn+fp)}",
        "Accuracy": f"{acc * 100:.2f}%",
        "Precision (PPV)": f"{prec * 100:.2f}%",
```

```
            "Recall (Sens.)": f"{rec * 100:.2f}%",
            "Specificity": f"{spec * 100:.2f}%",
            "F1-Score": f"{f1:.4f}",
            "ROC-AUC": f"{roc_auc:.4f}",
            "PR-AUC": f"{pr_auc:.4f}"
        })

    metrics_df = pd.DataFrame(metrics_list)
    print("\n📋 CLINICAL PERFORMANCE SUMMARY (Validation Cohort):")
    print(metrics_df.to_string(index=False))

    # 4. Generate Diagnostic Plots
    plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
    fig = plt.figure(figsize=(18, 14))

    # ------------------------------------------------------------
    # PLOT 1: Confusion Matrices for All 3 Classes
    # ------------------------------------------------------------
    for i, task in enumerate(task_names):
        ax = plt.subplot(2, 3, i + 1)
        cm = confusion_matrix(all_targets[:, i], all_preds[:, i], labels=[0, 1])

        # Annotations with counts and percentages
        cm_perc = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis] * 100
        labels_matrix = np.array([
            [f"{cm[0,0]}\n({cm_perc[0,0]:.1f}%)", f"{cm[0,1]}\n({cm_perc[0,1]:.1f}%)"],
            [f"{cm[1,0]}\n({cm_perc[1,0]:.1f}%)", f"{cm[1,1]}\n({cm_perc[1,1]:.1f}%)"]
        ])

        sns.heatmap(cm, annot=labels_matrix, fmt='', cmap="Blues", cbar=False, ax=ax,
                    xticklabels=["Negative (0)", "Positive (1)"],
                    yticklabels=["Negative (0)", "Positive (1)"],
                    annot_kws={"size": 11, "weight": "bold"})
        ax.set_title(f"Confusion Matrix: {task}", fontsize=13, fontweight='bold', pad=10)
        ax.set_xlabel("Predicted Label", fontsize=11)
        ax.set_ylabel("True Label", fontsize=11)

    # ------------------------------------------------------------
    # PLOT 2: Receiver Operating Characteristic (ROC) Curves
    # ------------------------------------------------------------
    ax_roc = plt.subplot(2, 3, 4)
    colors = ["#2b5c8f", "#d95f02", "#7570b3"]

    for i, (task, color) in enumerate(zip(task_names, colors)):
        fpr, tpr, _ = roc_curve(all_targets[:, i], all_probs[:, i])
        auc_val = roc_auc_score(all_targets[:, i], all_probs[:, i])
        ax_roc.plot(fpr, tpr, color=color, lw=2.2, label=f"{task} (AUC = {auc_val:.3f})")

    ax_roc.plot([0, 1], [0, 1], color='gray', linestyle='--', lw=1.5, label="Random Guess (0.50)")
    ax_roc.set_xlim([0.0, 1.0])
    ax_roc.set_ylim([0.0, 1.05])
    ax_roc.set_xlabel("False Positive Rate (1 - Specificity)", fontsize=11)
    ax_roc.set_ylabel("True Positive Rate (Recall / Sensitivity)", fontsize=11)
    ax_roc.set_title("Multi-Task ROC Curves", fontsize=13, fontweight='bold', pad=10)
    ax_roc.legend(loc="lower right", frameon=True, fontsize=10)
    ax_roc.grid(True, linestyle="--", alpha=0.5)

    # ------------------------------------------------------------
    # PLOT 3: Precision-Recall (PR) Curves
    # ------------------------------------------------------------
    ax_pr = plt.subplot(2, 3, 5)

    for i, (task, color) in enumerate(zip(task_names, colors)):
        prec_pts, rec_pts, _ = precision_recall_curve(all_targets[:, i], all_probs[:, i])
        ap_val = average_precision_score(all_targets[:, i], all_probs[:, i])
        ax_pr.plot(rec_pts, prec_pts, color=color, lw=2.2, label=f"{task} (AP = {ap_val:.3f})")

    ax_pr.set_xlim([0.0, 1.0])
    ax_pr.set_ylim([0.0, 1.05])
    ax_pr.set_xlabel("Recall (Sensitivity)", fontsize=11)
    ax_pr.set_ylabel("Precision (Positive Predictive Value)", fontsize=11)
    ax_pr.set_title("Precision-Recall (PR) Curves", fontsize=13, fontweight='bold', pad=10)
    ax_pr.legend(loc="upper right", frameon=True, fontsize=10)
    ax_pr.grid(True, linestyle="--", alpha=0.5)

    # ------------------------------------------------------------
    # PLOT 4: Comparison Bar Chart Across Diagnostic Metrics
```

```
# ------------------------------------------------------------
ax_bar = plt.subplot(2, 3, 6)

plot_metrics = ["Accuracy", "Recall", "Precision", "Specificity", "F1"]
bar_data = []

for i, task in enumerate(task_names):
    y_true = all_targets[:, i]
    y_prob = all_probs[:, i]
    y_pred = all_preds[:, i]
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()

    bar_data.append([
        accuracy_score(y_true, y_pred),
        recall_score(y_true, y_pred, zero_division=0),
        precision_score(y_true, y_pred, zero_division=0),
        tn / (tn + fp) if (tn + fp) > 0 else 0.0,
        f1_score(y_true, y_pred, zero_division=0)
    ])

bar_data = np.array(bar_data)  # (3, 5)
x = np.arange(len(plot_metrics))
width = 0.25

for i, (task, color) in enumerate(zip(task_names, colors)):
    ax_bar.bar(x + (i - 1) * width, bar_data[i], width, label=task, color=color, alpha=0.85)

ax_bar.set_xticks(x)
ax_bar.set_xticklabels(plot_metrics, fontsize=11, fontweight='bold')
ax_bar.set_ylim([0.0, 1.1])
ax_bar.set_ylabel("Score (0.0 to 1.0)", fontsize=11)
ax_bar.set_title("Metric Comparison by Condition", fontsize=13, fontweight='bold', pad=10)
ax_bar.legend(loc="lower right", frameon=True, fontsize=10)
ax_bar.grid(True, linestyle="--", alpha=0.5, axis='y')

plt.tight_layout(pad=3.0)
plt.show()
```

```
================================================================================
[EVALUATION] FULL CLINICAL DIAGNOSTIC METRICS & MULTI-PANEL PLOTS
================================================================================
✓ Loaded best global checkpoint from: /content/best_federated_mrnet.pth

📋 CLINICAL PERFORMANCE SUMMARY (Validation Cohort):
    Condition Total (Pos/Neg) Accuracy Precision (PPV) Recall (Sens.) Specificity F1-Score ROC-AUC PR-AUC
     Abnormal         95 / 25   79.17%          79.17%        100.00%       0.00%   0.8837  0.6524 0.8433
     ACL Tear         54 / 66   55.00%           0.00%          0.00%     100.00%   0.0000  0.5568 0.5085
Meniscus Tear         52 / 68   56.67%           0.00%          0.00%     100.00%   0.0000  0.4713 0.4198
```

Start coding or generate with AI.

